

Problem Set 6

Part 1

```
%% Part A
start_time = 100;
end_time = 400;

num_trials = length(R);
num_units = length(R(1).unit);
spikeCounts = zeros(num_units, num_trials);

for trial = 1:num_trials
    targetAppearsTime = R(trial).targetAppearsTime;
    start_window = targetAppearsTime + start_time;
    end_window = targetAppearsTime + end_time;
    for unit = 1:num_units
        spike = R(trial).unit(unit).spikeTimes;
        spikeCounts(unit, trial) = sum(spike > start_window & spike <=
end_window);
    end
end
disp('Spike Counts Example');
```

Spike Counts Example

```
disp(spikeCounts(1, :))
```

6 5 6 3 2 1 8 6 3 4 6 8 4 4 2 4 5

```
%% Part B
window_size = (abs(end_time) + abs(start_time)) * 0.001;

conditions = [R.conditionID];

cond_one = conditions(1);
cond_two = conditions(2);

cond_one_idx = conditions == cond_one;
cond_two_idx = conditions == cond_two;

avgRateOne = mean(spikeCounts(:, cond_one_idx), 2) / window_size;
avgRateTwo = mean(spikeCounts(:, cond_two_idx), 2) / window_size;

disp('Average spike counts (cond 16)');
```

Average spike counts (cond 16)

```
disp(avgRateOne)
```

10.8571
1.4286
0
0.2857
0.0714
0.6429
0.4286
1.7143
0.3571
4.3571
0
0.2857
2.5714
0.2143
6.5000
0
0
0.5000
0.0714
0.6429
0.6429
0
0.1429
2.7857
0.5000
1.2143
5.2857
0.4286
6.2857
3.6429
2.3571
0.3571
0.0714
0.5000
0.7857
1.8571
0.9286
0.9286
0.5714
0.7143
1.2857
0.4286
0.3571
6.4286
4.0000
10.2143
0.8571
2.1429
2.5000
0.7143
2.6429
12.5000
0.6429
1.0000
0.4286
1.2857
1.1429
0.9286
0.4286
1.9286
2.7143
5.4286
0.6429
4.2857

0.5000
6.0000
0.2857
0.4286
1.0000
1.3571
1.9286
0.7857
3.1429
0.7857
2.0000
2.0714
0.5714
0.7857
0.4286
0.3571
3.7143
0.5714
0.4286
0.6429
2.6429
8.9286
0.0714
0.8571
0.8571
2.9286
3.6429
4.8571
0.2143
6.0000
2.5000
4.2857
6.5000
2.4286
0.3571
6.0000
5.3571
3.1429
1.8571
0.7857
0.1429
1.2857
4.5000
8.5714
0.5714
8.7857
1.7143
0.2857
4.3571
9.0714
0.2143
0.3571
9.7143
0.2143
0.7857
2.2143
5.9286
0.3571
1.7143
3.2857
10.4286
0.2857
2.5000
6.3571

3.3571
7.5000
0.0714
2.3571
2.0714
2.1429
0.7857
5.7857
1.5714
4.0714
1.0714
0.9286
0.7857
6.2143
0.6429
0.7143
1.2143
0.9286
3.5714
1.7857
2.5714
0.1429
2.9286
0.5000
2.4286
12.6429
3.7143
3.6429
0.7143
8.7857
0.1429
4.2857
4.8571
1.5714
3.3571
0.4286
0.2143
0.2857
0.5000
4.0000
2.2143
4.5714
2.8571
0.4286
0.6429
0.5000
0.9286
8.0714
12.1429
0.9286
0.0714
1.3571
5.5000
1.2143
7.9286
1.0714
5.6429
7.8571
1.0000
3.2857
1.5000
0.6429
5.7857
4.0000

```
0.8571
4.7857
0.8571
5.3571
0.8571
1.8571
3.5000
```

```
disp('Average spike counts (cond 2)');
```

Average spike counts (cond 2)

```
disp(avgRateTwo)
```

```
6.9333
0.4000
0.1333
0.6667
0
0.8000
1.0000
0.6667
0.9333
3.9333
1.4000
0.2000
2.2667
0.0667
1.8667
0.0667
0.3333
1.1333
0.1333
2.0667
1.1333
0
0.0667
1.0000
0.2000
1.0667
5.1333
0.6000
4.2667
3.0000
1.2667
0.1333
0.1333
1.2667
0.3333
2.8000
1.6667
1.2667
0.4667
0.0667
1.4667
0.1333
0.7333
9.2667
5.8000
9.4000
0.8667
1.6667
2.0667
```

0.4000
3.0667
9.0667
0.8667
0.6000
0.5333
1.2667
0.8000
2.0000
0.8667
1.6000
2.2000
4.0000
0.7333
2.9333
1.5333
6.2667
0.0667
0.0667
0.6667
2.4667
3.4667
0.5333
2.6000
0.6000
0.8000
1.8667
1.0667
1.2667
0.6000
0.1333
4.9333
0.5333
0.1333
0.4667
4.0667
9.6000
0
1.1333
0.2667
2.3333
2.2667
5.0667
0.2667
5.2000
1.8000
4.4667
3.8000
2.8667
0.2667
6.5333
5.4667
3.4000
1.8000
1.0667
0
0.7333
3.5333
6.3333
0.6000
8.1333
0.7333
0.1333
4.2000

8.9333
0.2000
0.7333
10.2000
0.1333
1.2667
1.1333
5.4667
0.4000
1.7333
3.8667
11.6000
0.4000
2.9333
5.4667
1.0667
3.8667
0.0667
2.7333
1.8000
0.9333
0.8000
5.4000
2.0000
4.1333
1.7333
1.2000
0.4000
4.8667
0.3333
1.6667
1.5333
0.8667
3.6667
1.4667
3.0000
1.0000
2.2667
0.4000
2.6667
13.8667
2.8667
3.8000
1.4000
8.0000
0.4667
6.2000
4.8667
1.6000
2.9333
0.2000
0.4667
0
0.5333
4.1333
5.1333
3.9333
1.2000
0.3333
0.8667
0.2667
1.2667
8.2000
10.1333

```

0.2000
0.0667
1.0667
6.0667
0.9333
6.4667
1.2000
5.5333
7.8667
1.3333
3.1333
1.2000
0.6667
6.3333
2.7333
1.2000
4.4000
0.8000
4.4000
1.1333
1.7333
3.4667

```

```
%% Part C
```

```
diffRate = abs(avgRateOne - avgRateTwo);
```

```
tunedNeuronMask = diffRate >= 2;
```

```
tunedNeuronIdx = find(tunedNeuronMask);
```

```
disp('Tuned units:');
```

```
Tuned units:
```

```
fprintf('\n%5s %14s %14s %8s\n', 'Unit', ...
        sprintf('Cond%d (sp/s)', cond_one), ...
        sprintf('Cond%d (sp/s)', cond_two), '|Diff|');
```

```
Unit    Cond16 (sp/s)    Cond25 (sp/s)    |Diff|
```

```
for k = 1:length(tunedNeuronIdx)
    u = tunedNeuronIdx(k);

    fprintf('%5d %14.2f %14.2f %8.2f\n', ...
            u, avgRateOne(u), avgRateTwo(u), diffRate(u));
end
```

```

1      10.86      6.93      3.92
15      6.50      1.87      4.63
29      6.29      4.27      2.02
44      6.43      9.27      2.84
52     12.50      9.07      3.43
97      6.50      3.80      2.70
108     8.57      6.33      2.24
129     3.36      1.07      2.29
130     7.50      3.87      3.63

```


169	2.21	5.13	2.92
177	12.14	10.13	2.01

Part 2

```
X = spikeCounts(tunedNeuronIdx, :);
y = conditions';

mdl = fitcnb(X, y, 'DistributionNames', 'kernel');

cv = crossval(mdl, 'KFold', 10);
lossCV = kfoldLoss(cv);
accCV = (1 - lossCV) * 100;

fprintf('10-fold CV misclassification rate : %.4f\n', lossCV);
```

10-fold CV misclassification rate : 0.1724

```
fprintf('10-fold CV accuracy : %.2f%%\n', accCV);
```

10-fold CV accuracy : 82.76%

Part 3

```
%% Part A
y_bin = double(conditions' == cond_two);
X_tuned = spikeCounts(tunedNeuronIdx, :);
cv3 = cvpartition(y_bin, 'KFold', 10);

correctLR = 0;
for fold = 1:cv3.NumTestSets
    X_train = X_tuned(cv3.training(fold), :);
    y_train = y_bin(cv3.training(fold));
    X_test = X_tuned(cv3.test(fold), :);
    y_test = y_bin(cv3.test(fold));

    b = glmfit(X_train, y_train, 'binomial', 'link', 'logit');

    pHat = glmval(b, X_test, 'logit');
    yPred = double(pHat >= 0.5);

    correctLR = correctLR + sum(yPred == y_test);
end
```

Warning: Iteration limit reached.
Warning: The estimated coefficients perfectly separate failures from successes. This means the theoretical best estimates are not finite. For the fitted linear combination XB of the predictors,

the sample proportions P of $Y=N$ in the data satisfy:

$XB < 0.177009$: $P=0$

$XB > 0.177009$: $P=1$

To get proper estimates, fit with 'LikelihoodPenalty' set to 'jeffreys-prior'.

Warning: Iteration limit reached.

Warning: The estimated coefficients perfectly separate failures from successes. This means the theoretical best estimates are not finite. For the fitted linear combination XB of the predictors, the sample proportions P of $Y=N$ in the data satisfy:

$XB < 0.11433$: $P=0$

$XB > 0.11433$: $P=1$

To get proper estimates, fit with 'LikelihoodPenalty' set to 'jeffreys-prior'.

```
accLR = correctLR / num_trials * 100;
```

```
fprintf('10-fold CV accuracy (Logistic Regression): %.2f%%\n', accLR);
```

```
10-fold CV accuracy (Logistic Regression): 77.59%
```

I think the fitting procedure produces warnings because there are not many trials and features which overparameterizes the model. This makes the data nearly perfectly separable in the high dimensional feature space, making the likelihood estimates go to infinity. The model has too many weights relative to the number of training examples as a result and cannot return a set of coefficients that will maximize the likelihood, which glmfit warns of.

%% Part B

```
cvOuter = cvpartition(y_bin, 'HoldOut', 0.2);
```

```
X_fit = X_tuned(cvOuter.training, :);
```

```
y_fit = y_bin(cvOuter.training);
```

```
X_test = X_tuned(cvOuter.test, :);
```

```
y_test = y_bin(cvOuter.test);
```

```
[B, FitInfo] = lassoglm(X_fit, y_fit, 'binomial', 'Alpha', 0.1, 'CV', 10);
```

```
lambdaIdx = FitInfo.IndexMinDeviance;
```

```
B_best = B(:, lambdaIdx);
```

```
intercept = FitInfo.Intercept(lambdaIdx);
```

```
logits = X_test * B_best + intercept;
```

```
pHat = 1 ./ (1 + exp(-logits));
```

```
yPred = double(pHat >= 0.5);
```

```
accEN = sum(yPred == y_test) / length(y_test) * 100;
```

```
fprintf('Best Lambda (min CV deviance) : %.4f\n',  
FitInfo.LambdaMinDeviance);
```

```
Best Lambda (min CV deviance) : 0.0641
```

```
fprintf('Non-zero coefficients : %d / %d\n', sum(B_best ~= 0),
length(B_best));
```

```
Non-zero coefficients : 11 / 11
```

```
fprintf('Test set accuracy (ElasticNet): %.2f%%\n', accEN);
```

```
Test set accuracy (ElasticNet): 81.82%
```

This regularization makes the warnings go away because the L2 component zeros out or lowers some of the coefficients towards zero. This stops the coefficients from going towards infinity like they do in the previous part. Increases in the coefficients is penalized, which leads to a finite solution and no warning.

Part 4

```
%% Part A
```

```
y_svm = ones(num_trials, 1);
y_svm(conditions' == cond_one) = -1;
```

```
mdlSVMlin = fitcsvm(X_tuned, y_svm, 'KernelFunction', 'linear');
```

```
cvSVMlin = crossval(mdlSVMlin, 'KFold', 10);
accSVMlin = (1 - kfoldLoss(cvSVMlin)) * 100;
```

```
fprintf('10-fold CV accuracy (Linear SVM): %.2f%%\n', accSVMlin);
```

```
10-fold CV accuracy (Linear SVM): 75.86%
```

Logistic regression works by finding the maximum likelihood decision boundary. In the example in Part 3A, the likelihood goes to infinity when the classes are perfectly separable, leading to the warning we received. A linear SVM maximizes the margin between the two nearest points, not the likelihood. Therefore, even with perfect separability, the margin will have a finite maximum and finite number of coefficient solutions. This makes it so that this fit does not have the same trouble as logistic regression.

```
%% Part B:
```

```
mdlSVMrbf = fitcsvm(X_tuned, y_svm, ...
    'KernelFunction', 'rbf', ...
    'OptimizeHyperparameters', 'auto', ...
    'HyperparameterOptimizationOptions', ...
    struct('AcquisitionFunctionName', 'expected-improvement-plus', ...
        'ShowPlots', false, ...
        'Verbose', 0));
```

```
cvSVMrbf = crossval(mdlSVMrbf, 'KFold', 10);
accSVMrbf = (1 - kfoldLoss(cvSVMrbf)) * 100;
```

```
fprintf('10-fold CV accuracy (RBF SVM): %.2f%%\n', accSVMrbf);
```

```
10-fold CV accuracy (RBF SVM): 84.48%
```

The model performs better than the others that we have tested. I think this is because the RBF kernel maps the data into an infinite dimensional feature space. This makes it better at fitting to nonlinear boundaries than the logistic regression and linear SVM.

Part 5

```
SD_MS = 30;  
BIN_MS = 1;  
N_PCS = 6;  
  
%% Parts A-D  
epoch_AD = [-150, 200];  
[M_AD, eigvals_AD, ~] = fitDynamics(R, epoch_AD, SD_MS, BIN_MS, N_PCS);  
  
fprintf('(epoch %d to %d ms) \n', epoch_AD);
```

```
(epoch -150 to +200 ms)
```

```
fprintf('M matrix estimated (%dx%d)\n', size(M_AD,1), size(M_AD,2));
```

```
M matrix estimated (6x6)
```

```
fprintf('Eigenvalues of M:\n');
```

```
Eigenvalues of M:
```

```
disp(eigvals_AD);
```

```
-0.0042 + 0.0068i  
-0.0042 - 0.0068i  
-0.0020 + 0.0031i  
-0.0020 - 0.0031i  
-0.0059 + 0.0000i  
-0.0046 + 0.0000i
```

```
%% Part E  
epoch_E = [-150, -50];  
[M_E, eigvals_E, ~] = fitDynamics(R, epoch_E, SD_MS, BIN_MS, N_PCS);  
  
fprintf('(epoch %d to %d ms)\n', epoch_E);
```

```
(epoch -150 to -50 ms)
```

```
fprintf('Eigenvalues of M:\n');
```

```
Eigenvalues of M:
```

```
disp(eigvals_E);
```

```
-0.0317 + 0.0000i  
-0.0165 + 0.0000i  
-0.0151 + 0.0000i  
-0.0075 + 0.0000i  
-0.0084 + 0.0018i  
-0.0084 - 0.0018i
```

```
%% Part D + E plot
```

```
figure;
```

```
hold on;
```

```
grid on;
```

```
axis equal;
```

```
% Full window (blue)
```

```
scatter(real(eigvals_AD), imag(eigvals_AD), 80, 'b', 'filled', ...  
        'DisplayName', sprintf('Full window (%d to +%d ms)', epoch_AD));
```

```
% Preparatory window (red)
```

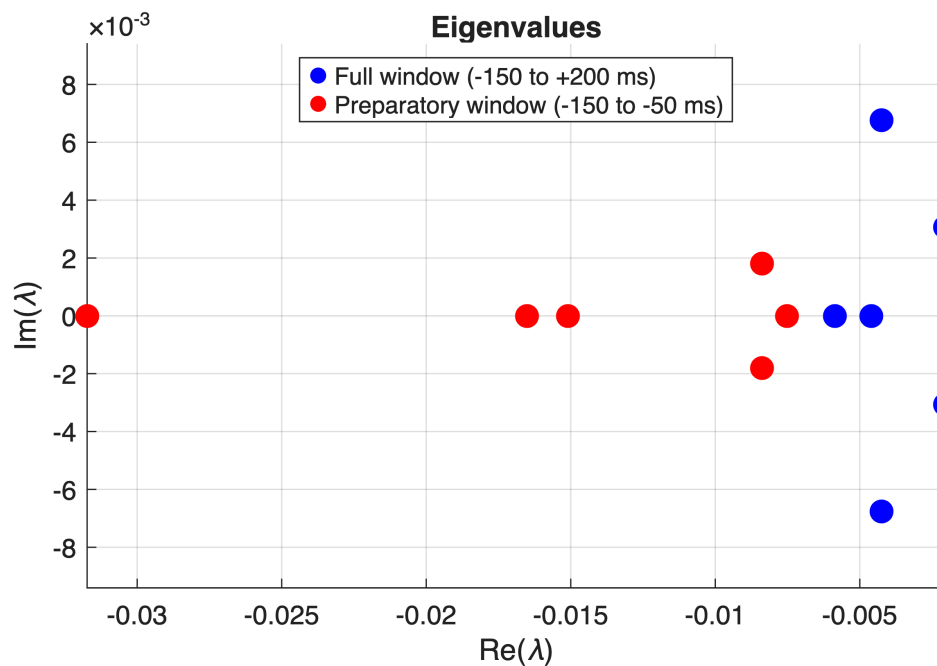
```
scatter(real(eigvals_E), imag(eigvals_E), 80, 'r', 'filled', ...  
        'DisplayName', sprintf('Preparatory window (%d to +%d ms)',  
epoch_E));
```

```
xlabel('Re(\lambda)');
```

```
ylabel('Im(\lambda)');
```

```
title('Eigenvalues');
```

```
legend('Location', 'best');
```



The eigenvalues in the preparatory window have lower real values than the full window eigenvalues. The full window eigenvalues also have a larger larger distribution in the imaginary component when compared to the preparatory window. The preparatory window eigenvalues show that the system is being pulled to a stable fixed point since different point are converging towards the center of the graph. This is analogous to the green trajectories we saw in class which spiral inward. The full movement window parts are not contracting with larger imaginary parts. This is what would make the dynamics more rotational which matches the red trajectories we saw in class. The preparatory activity stabilizes the initial coniditons before driving the movement we see in the full window.

Helper functions

```
function [M, eigvals, x] = fitDynamics(R, epoch_ms, sd_ms, bin_ms, n_pcs)
    nTrials = length(R);
    nUnits = length(R(1).unit);
    t_start = epoch_ms(1);
    t_stop = epoch_ms(2);

    binEdges = t_start : bin_ms : t_stop;
    nBins = length(binEdges) - 1;

    uniqueConds = unique([R.conditionID]);
    nConditions = length(uniqueConds);

    % Part A: bin spikes aligned to movement onset
    data = zeros(nBins, nTrials, nUnits);
    for tr = 1:nTrials
        for u = 1:nUnits
            alignedTimes = R(tr).unit(u).spikeTimes(:) -
R(tr).moveOnsetTime;
            [N, ~] = histcounts(alignedTimes, binEdges);
            data(:, tr, u) = N;
        end
    end

    smoothData = gaussFilt1(data, sd_ms);

    trialAvg = zeros(nBins, nConditions, nUnits);
    for cond = 1:nConditions
        condMask = [R.conditionID] == uniqueConds(cond);
        condTrials = find(condMask);
        trialAvg(:, cond, :) = mean(smoothData(:, condTrials, :), 2) * 100;
    end

    neuralData = reshape(trialAvg, nBins * nConditions, nUnits);

    for n = 1:size(neuralData, 2)
        neuralData(:, n) = neuralData(:, n) ./ (range(neuralData(:, n)) +
5);
```

```
end

% PCA
[~, score, ~] = pca(neuralData);
x = score(:, 1:n_pcs);

% Part B: finite differences
xDot = diff(x);

% Part C: regression using left endpoint
xTrunc = x(1:end-1, :);
M = (xTrunc \ xDot)';

% Part D: eigenvalues
eigvals = eig(M);
end
```